Aprendizaje de Máquina
Sesión 20: Aprendizaje de máquina para textos

Angel Rodríguez

Práctica #6: modificar la representación

Entrega: Martes 20 de Octubre.

Nuevo experimento

Repetir el agrupamiento sustituyendo TF-IDF por otra representación de los documentos.

1. Utilizar `CountVectorizer` para representar frecuencias de términos.
2. Normalizar los vectores mediante norma L_2.
3. Aplicar nuevamente K-means.
4. Comparar la silueta, la curva de inercia y la interpretación de los clusters.

Preguntas finales

¿La representación de los documentos influye en los grupos encontrados?

¿Podríamos utilizar embeddings en lugar de TF-IDF?

Resumen

El agrupamiento de textos requiere dos pasos fundamentales:

$$\boxed{\text{Texto} \longrightarrow \text{representación vectorial} \longrightarrow \text{clustering}}$$

Por ejemplo:

$$\boxed{\text{documentos} \rightarrow \text{TF-IDF} \rightarrow K\text{-means} \rightarrow \text{clusters}}$$

Las ideas principales son:

- cada documento se representa como un vector;
- la similitud coseno permite comparar vectores de texto;
- K-means agrupa vectores alrededor de centroides;
- los clusters deben interpretarse posteriormente.

Siguiente paso: clasificación de textos

Hasta ahora:

documentos sin etiquetas $\longrightarrow$ clusters

¿Qué ocurre si conocemos una etiqueta para cada documento?

reseña positiva reseña negativa	o	deportes finanzas tecnología

Entonces podemos utilizar **aprendizaje supervisado**.

4.4 Modelos de clasificación de texto

En el tema anterior:

$$\boxed{\text{documentos sin etiquetas} \longrightarrow \text{clustering}}$$

Ahora disponemos de ejemplos etiquetados:

$$(d_i, y_i).$$

Queremos aprender una función

$$\boxed{f(x) \longrightarrow \hat{y}}$$

que pueda asignar una clase a un **documento nuevo**.

Ejemplos de clasificación de texto

Algunos problemas pueden formularse como:

$$\text{documento} \longrightarrow \text{clase}.$$

Por ejemplo:

- análisis de sentimiento:

$$\text{reseña} \to \{\text{positiva}, \text{negativa}\};$$

- detección de spam:

$$\text{correo} \to \{\text{spam}, \text{no spam}\};$$

- clasificación temática:

$$\text{noticia} \to \{\text{deportes}, \text{finanzas}, \text{tecnología}, \ldots\}.$$

De texto a clasificación

El clasificador no recibe directamente una cadena de texto.

Primero construimos una representación numérica:

$$\boxed{\text{texto} \longrightarrow \text{representación} \longrightarrow x \longrightarrow \text{clasificador} \longrightarrow \hat{y}}$$

Por ejemplo:

$$\boxed{\text{texto} \xrightarrow{\text{TF-IDF}} x \xrightarrow{\text{Regresión logística}} \hat{y}.}$$

La representación y el clasificador forman parte del mismo **pipeline**.

Ejemplo: clasificación de sentimiento

Supongamos que queremos clasificar una reseña como

$$y \in \{0, 1\},$$

donde

$$0 = \text{negativa}, \qquad 1 = \text{positiva}.$$

Ejemplo de características por utilizar:

$$x = \begin{bmatrix} \#\text{ palabras positivas} \\ \#\text{ palabras negativas} \\ 1[\text{aparece “no”}] \\ \#\text{ pronombres} \\ 1[\text{aparece “!”}] \\ \log(\text{longitud}) \end{bmatrix}.$$

Características manuales vs. representación automática

El ejemplo anterior utiliza

características diseñadas manualmente.

Pero podemos utilizar directamente una representación como

$$x = [\text{TF-IDF}(t_1, d), \ldots, \text{TF-IDF}(t_p, d)].$$

Entonces:

cada término del vocabulario $\longrightarrow$ una característica

y la regresión logística puede aprender cuáles términos son útiles para distinguir las clases.

Regresión logística

A partir de un vector de características x, calculamos primero

$$z = w^\top x + b.$$

Luego aplicamos la función sigmoide:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

para obtener

$$\boxed{\hat{y} = P(y = 1 \mid x) = \sigma(w^\top x + b).}$$

La salida es una probabilidad

Para clasificación binaria,

$$\hat{y} = P(y = 1 \mid x).$$

Por tanto,

$$P(y = 0 \mid x) = 1 - \hat{y}.$$

Por ejemplo:

$$\hat{y} = 0{,}92$$

significa que el modelo asigna

$$P(\text{positiva} \mid x) = 0{,}92.$$

La regresión logística no produce inicialmente una clase:

produce una probabilidad.

De probabilidad a clase

Con el umbral estándar de 0,5:

$$\hat{y}_{\text{clase}} = \begin{cases} 1, & P(y = 1 \mid x) > 0{,}5, \\ 0, & \text{en otro caso.} \end{cases}$$

Entonces:

$$\boxed{P(y = 1 \mid x) \xrightarrow{\text{umbral}} \text{clase}}$$

El umbral puede modificarse dependiendo del problema y del costo relativo de los diferentes tipos de error.

¿Qué aprende el modelo?

Cada característica tiene un peso:

$$w = [w_1, w_2, \ldots, w_p].$$

En clasificación binaria:

$$\boxed{w_j > 0 \Rightarrow \text{evidencia hacia la clase 1}}$$

$$\boxed{w_j < 0 \Rightarrow \text{evidencia hacia la clase 0}}$$

Además,

$$|w_j| \text{ grande}$$

indica una mayor influencia de esa característica.

Interpretación en clasificación de sentimiento

Si las características corresponden a palabras, podríamos aprender pesos como

Palabra	w_j
excellent	2,8
amazing	2,1
enjoyable	1,7
boring	–2,0
awful	–3,1

Así:

$$\text{excellent} \Rightarrow \text{evidencia positiva},$$

mientras que

$$\text{awful} \Rightarrow \text{evidencia negativa}.$$

TF-IDF + Regresión logística

Para un documento d:

$$x_d = [x_1, \ldots, x_p],$$

donde

$$x_j = \text{tf-idf}(t_j, d).$$

La regresión logística calcula

$$z = w_1 x_1 + \cdots + w_p x_p + b.$$

Por tanto, cada palabra contribuye de acuerdo con

$$\boxed{\text{importancia en el documento} \times \text{peso aprendido}}$$

$$x_j w_j.$$

¿Cómo aprendemos los pesos?

Disponemos de un conjunto de entrenamiento

$$\mathcal{D} = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}.$$

Queremos encontrar

$$w, b$$

que produzcan probabilidades altas para las **clases correctas**.

Para hacerlo necesitamos definir una

función de pérdida.

Entropía cruzada

Para clasificación binaria:

$$\boxed{L_{CE}(\hat{y}, y) = -\left[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\right]}$$

Si $y = 1$:

$$L_{CE} = -\log(\hat{y}),$$

mientras que si $y = 0$:

$$L_{CE} = -\log(1-\hat{y}).$$

Una predicción correcta y segura produce **pérdida pequeña**.

Interpretación de la pérdida

Supongamos que la clase correcta es

$$y = 1.$$

Si el modelo predice

$$\hat{y} = 0{,}99,$$

entonces

$$-\log(0{,}99) \approx 0.$$

Pero si predice

$$\hat{y} = 0{,}01,$$

entonces

$$-\log(0{,}01) \quad \text{es grande.}$$

Predicciones incorrectas y muy seguras $\Rightarrow$ gran penalización.

Entrenamiento del modelo

El objetivo es minimizar la pérdida promedio:

$$J(w, b) = \frac{1}{n} \sum_{i=1}^{n} L_{CE}\left(\hat{y}^{(i)}, y^{(i)}\right)$$

con

$$\hat{y}^{(i)} = \sigma\left(w^{\top} x^{(i)} + b\right).$$

Los parámetros se ajustan mediante métodos de optimización como

descenso de gradiente.

¿Qué ocurre con más de dos clases?

No todos los problemas son binarios.

Por ejemplo:

$$y \in \{\text{positivo}, \text{neutral}, \text{negativo}\}.$$

O:

$$y \in \{\text{deportes}, \text{finanzas}, \text{tecnología}, \text{política}\}.$$

Para $K > 2$ clases utilizamos

regresión logística multinomial

también llamada

softmax regression.

Un conjunto de pesos para cada clase

En clasificación binaria teníamos un vector:

$$w.$$

Para K clases tenemos

$$w_1, w_2, \ldots, w_K.$$

Podemos reunirlos en una matriz:

$$W = \begin{bmatrix} - & - & - & w_1^\top & - & - \\ - & - & - & w_2^\top & - & - \\ & & & \vdots & & \\ - & - & - & w_K^\top & - & - \end{bmatrix}$$

Cada clase recibe su propio score:

$$z_k = w_k^\top x + b_k.$$

Softmax

Los scores

$$z_1, \ldots, z_K$$

se transforman en probabilidades mediante

$$\boxed{P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$$

de modo que

$$0 \leq P(y = k \mid x) \leq 1$$

y

$$\sum_{k=1}^{K} P(y = k \mid x) = 1.$$

Ejemplo multiclase

Para una noticia, el modelo podría producir

Clase	$P(y = k \mid x)$
deportes	0,08
finanzas	0,75
tecnología	0,12
otros	0,05

La predicción es

$$\boxed{\hat{y} = \arg\max_k P(y = k \mid x)}$$

por lo que en este ejemplo

$$\hat{y} = \text{finanzas}.$$

Pérdida para varias clases

Representamos la etiqueta correcta como un vector *one-hot*:

$$y = [0, \ldots, 1, \ldots, 0].$$

La salida del modelo es

$$\hat{y} = [\hat{y}_1, \ldots, \hat{y}_K].$$

La entropía cruzada multiclase es

$$\boxed{L_{CE} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)}$$

y, como solo una entrada de y vale 1,

$$L_{CE} = -\log P(\text{clase correcta} \mid x).$$

Implementación con Scikit-Learn

Podemos construir todo el proceso mediante un pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

El pipeline aprende tanto

$$\text{texto} \rightarrow \text{TF-IDF}$$

como

$$\text{TF-IDF} \rightarrow \text{clase}.$$

Probabilidades de clase

También podemos obtener las probabilidades:

```
model.predict_proba([
    "The movie was excellent"
])
```

Conceptualmente:

Clase	Probabilidad
negativa	0,08
positiva	0,92

y por tanto

$$\hat{y} = \text{positiva}.$$

Dos decisiones diferentes

En clasificación de texto elegimos al menos dos cosas:

Representación

y

Clasificador.

Por ejemplo:

$$\text{CountVectorizer} + \text{LogisticRegression}$$

o

$$\text{TfidfVectorizer} + \text{LogisticRegression}.$$

Cambiar la representación puede modificar sustancialmente el resultado aun cuando **el clasificador sea el mismo**.

Embeddings como entrada

En lugar de utilizar directamente un vector disperso TF-IDF, podemos utilizar representaciones densas:

$$\boxed{\text{texto} \longrightarrow \text{embeddings} \longrightarrow \text{clase}}$$

La diferencia conceptual es:

TF-IDF	Embeddings
representación dispersa	representación densa

El objetivo sigue siendo el mismo:

$$P(y \mid \text{texto}).$$

Práctica 7: clasificación de documentos

Utilizaremos las cuatro categorías de **20 Newsgroups** de la clase anterior.

1. Cargar los documentos y sus etiquetas.
2. Separar los datos en entrenamiento y prueba.
3. Construir un `Pipeline` con `TfidfVectorizer` y `LogisticRegression`.
4. Entrenar el modelo y evaluar su precisión en el conjunto de prueba.
5. Examinar predicciones correctas e incorrectas.
6. Identificar las palabras con mayor influencia en cada categoría.
7. **Buscar una noticia en Internet** e intentar predecir su categoría utilizando el modelo.

Pregunta

¿Podemos utilizar los textos etiquetados para clasificar automáticamente documentos nuevos?

Paso 1: cargar los documentos

Utilizamos las mismas cuatro categorías de la práctica anterior.

```
from sklearn.datasets import fetch_20newsgroups

categorias = ["sci.space","rec.sport.baseball","comp.graphics","talk.
    politics.mideast"]

datos = fetch_20newsgroups(subset="train",categories=categorias,remove=("
    headers", "footers", "quotes"),shuffle=True,random_state=42)

documentos = datos.data
etiquetas = datos.target

print("Documentos:", len(documentos))
print("Categorias:", datos.target_names)
```

Diferencia con la clase pasada:

Ahora utilizamos las etiquetas reales (`datos.target`) para entrenar el modelo.

Paso 2: inspeccionar los datos

Cada documento tiene una categoría conocida.

```
print(documentos[11][:500])

print("Etiqueta numerica:", etiquetas[11])

print("Categoria:",datos.target_names[etiquetas[11]])
```

Observaciones

- `documentos` contiene los textos.
- `etiquetas` contiene sus categorías.
- Las categorías están codificadas mediante números enteros.

Paso 3: entrenamiento y prueba

Dividimos los documentos en dos conjuntos: 80 % para entrenamiento y 20 % para prueba.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    documentos,
    etiquetas,
    test_size=0.2,
    random_state=42,
    stratify=etiquetas
)

print("Entrenamiento:", len(X_train))
print("Prueba:", len(X_test))
```

Paso 4: construir el Pipeline

Combinamos TF-IDF y regresión logística para automatizar la transformación de textos y su clasificación:

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer(
        stop_words="english",
        min_df=5,
        max_df=0.8,
        max_features=5000,
        norm="l2"
    )),
    ("classifier", LogisticRegression(
        max_iter=1000,
        random_state=42
    ))
])

model
```

Paso 5: entrenar y predecir

Entrenamos el modelo y obtenemos predicciones para los documentos de prueba.

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicciones:")
print(y_pred[:10])

print("Etiquetas reales:")
print(y_test[:10])
```

¿Qué hace `fit`?

1. Aprende el vocabulario y los pesos IDF con los documentos de entrenamiento.
2. Transforma los documentos en vectores.
3. Ajusta los coeficientes de la regresión logística.

Paso 6: evaluar el clasificador

Calculamos la exactitud (*accuracy*):

$$\text{Accuracy} = \frac{\text{Predicciones correctas}}{\text{Total de predicciones}}.$$

```
from sklearn.metrics import accuracy_score,classification_report

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.3f}")

print( classification_report(y_test,y_pred,target_names=datos.
    target_names) )
```

Pregunta

¿El modelo clasifica todas las categorías con la misma precisión?

Resultados de la clasificación

Accuracy: 0.868 (86.8 %)

Categoría	Precision	Recall	F1	Support
comp.graphics	0.90	0.85	0.88	117
rec.sport.baseball	0.78	0.94	0.85	119
sci.space	0.87	0.82	0.84	119
talk.politics.mideast	0.94	0.86	0.90	113

Interpretación de las métricas

- **Precision:** de los documentos predichos en una categoría, proporción correcta.
- **Recall:** de los documentos reales de una categoría, proporción identificada.
- **F1-score:** media armónica de precision y recall.
- **Support:** número de documentos reales de cada categoría.

Observación: Política tiene el mayor F1 (0.90); béisbol tiene el mayor recall (0.94), pero la menor precision (0.78).

Paso 7: matriz de confusión

Visualizamos las categorías reales y predichas.

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

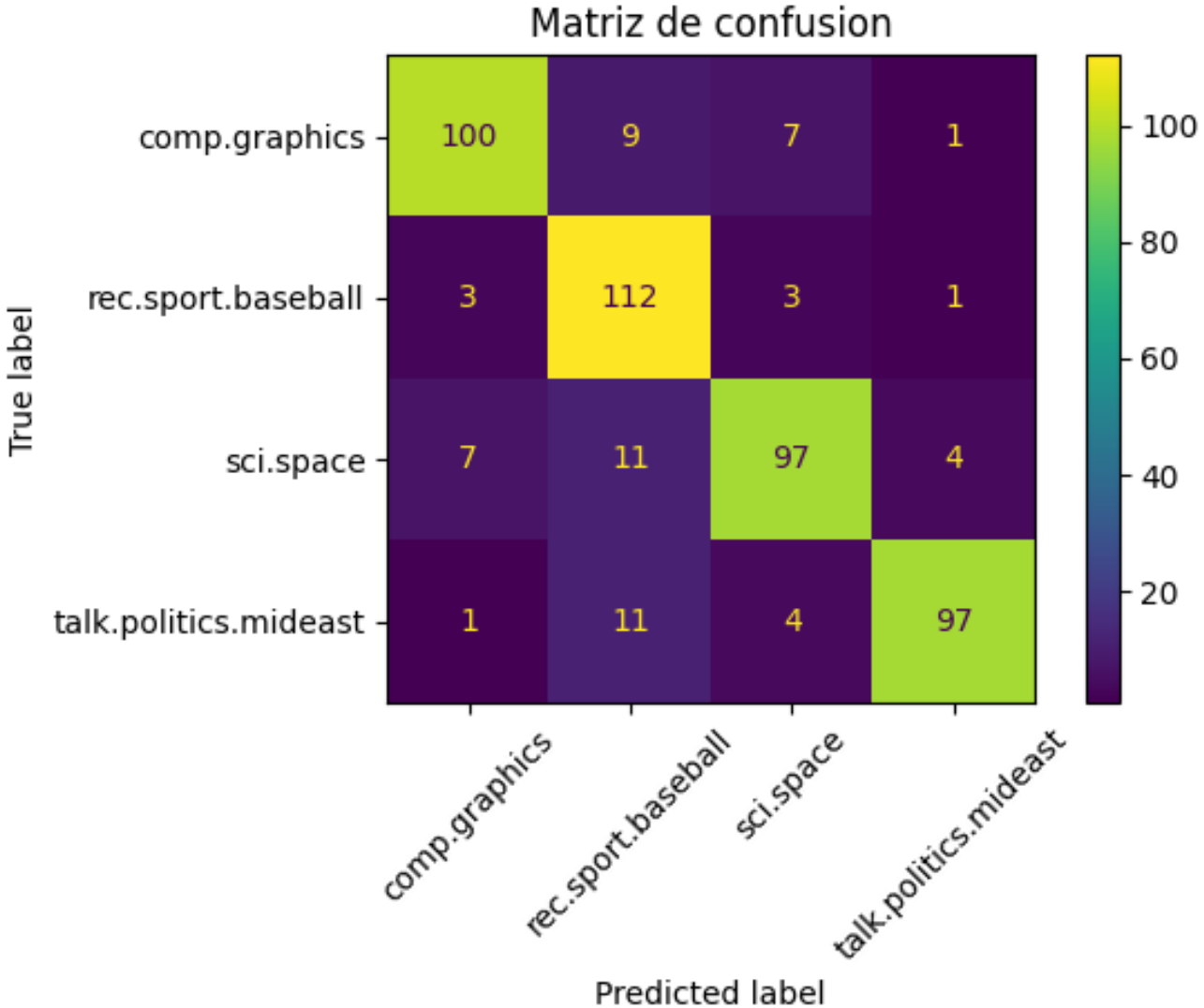
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=datos.target_names,
    xticks_rotation=45
)

plt.title("Matriz de confusion")
plt.tight_layout()
plt.show()
```

Interpretación

- Diagonal: predicciones correctas.
- Fuera de la diagonal: confusiones entre categorías.

Matriz de confusión



Paso 8: predicciones correctas e incorrectas

Identificamos los documentos bien y mal clasificados.

```
import numpy as np

correctos = np.where(y_pred == y_test)[0]
incorrectos = np.where(y_pred != y_test)[0]

print("Correctos:", len(correctos))
print("Incorrectos:", len(incorrectos))

def mostrar_predicciones(indices, cantidad=3):
    for i in indices[:cantidad]:
        print("\n" + "=" * 50)
        print("Real:", datos.target_names[y_test[i]])
        print("Predicha:", datos.target_names[y_pred[i]])
        print(X_test[i][:500])
```

Paso 9: examinar los errores

Mostramos algunos documentos correctamente clasificados y otros incorrectamente clasificados.

```
print("PREDICCIONES CORRECTAS")
mostrar_predicciones(correctos)
```

```
print("\nPREDICCIONES INCORRECTAS")
mostrar_predicciones(incorrectos)
```

Preguntas

1. ¿Qué características tienen los textos clasificados correctamente?
2. ¿Por qué algunos documentos se confunden?
3. ¿Existen textos ambiguos o demasiado cortos?

Paso 10: inspeccionar los coeficientes

Recuperamos los términos de TF-IDF y los coeficientes de la regresión logística.

`.named_steps` nos permite acceder a cada componente del pipeline mediante el nombre que le asignamos. Es similar a un diccionario.

```
tfidf = model.named_steps["tfidf"]
clasificador = model.named_steps["classifier"]

terminos = tfidf.get_feature_names_out()
coeficientes = clasificador.coef_

print("Numero de terminos:", len(terminos))
print("Dimensiones:", coeficientes.shape)
```

Interpretación

Cada fila de `coeficientes` contiene los pesos asociados a una categoría.
Un peso positivo grande favorece esa categoría; un peso negativo grande en magnitud la desfavorece.

Paso 11: palabras más influyentes

Identificamos las palabras con pesos positivos y negativos más grandes para cada categoría.

```
for j, clase in enumerate(clasificador.classes_):
    pesos = coeficientes[j]

    positivos = np.argsort(pesos)[-10:][::-1]
    negativos = np.argsort(pesos)[:10]

    print("\nCategoria:", datos.target_names[clase])

    print("Pesos positivos:")
    print(", ".join(terminos[positivos]))

    print("Pesos negativos:")
    print(", ".join(terminos[negativos]))
```

Pregunta

¿Qué palabras ayudan a distinguir cada categoría?

Paso 12: clasificar una noticia nueva

Busca una noticia en inglés relacionada con alguno de los cuatro temas. Copia un fragmento y utiliza el modelo para predecir su categoría.

```
noticia = """
NASA scientists are preparing a new mission
to study the surface of Mars. The spacecraft
will collect data about the planet's atmosphere.
"""

prediccion = model.predict([noticia])[0]

print("Categoria predicha:")
print(datos.target_names[prediccion])
```

Pregunta

¿La categoría predicha coincide con el tema de la noticia?

Paso 13: probabilidades de clasificación

Examinamos las probabilidades estimadas para cada categoría.

```
probabilidades = model.predict_proba([noticia])[0]

for clase, probabilidad in zip(clasificador.classes_,probabilidades):
    print(
        datos.target_names[clase],
        f"{probabilidad:.3f}"
    )
```

Importante

El modelo siempre elige una de las cuatro categorías, incluso si la noticia pertenece a un tema diferente.
Las probabilidades no garantizan que la predicción sea correcta.

Discusión: K-means vs. regresión logística

En ambas prácticas utilizamos los mismos documentos y la representación TF-IDF.

- **K-means (no supervisado):** agrupa documentos sin utilizar sus categorías.
- **Regresión logística (supervisado):** aprende a predecir categorías a partir de documentos etiquetados.

Preguntas finales

1. ¿Qué categorías fueron más fáciles de clasificar?
2. ¿Qué palabras ayudan a distinguirlas?
3. ¿Qué ocurrió con la noticia nueva?
4. ¿Qué ventajas tiene utilizar etiquetas durante el entrenamiento?

Resumen

La clasificación de texto transforma el problema

$$\boxed{\text{texto} \longrightarrow \text{etiqueta}}$$

en

$$\boxed{\text{texto} \longrightarrow x \longrightarrow P(y \mid x) \longrightarrow \hat{y}.}$$

En este tema vimos:

- representación de documentos mediante características;
- regresión logística binaria;
- entropía cruzada;
- clasificación multiclase con softmax;
- embeddings y clasificadores neuronales;
- fine-tuning de modelos preentrenados.

Siguiente paso: ¿qué modelo elegimos?

Podemos construir distintos modelos:

$$\text{CountVectorizer} + \text{LogisticRegression},$$

$$\text{TF-IDF} + \text{LogisticRegression},$$

o incluso modelos basados en embeddings.

Pero:

¿Cómo sabemos cuál generaliza mejor?

Necesitamos utilizar

$$\boxed{\text{métricas} + \text{validación} + \text{comparación de modelos.}}$$

4.5 Evaluación y selección de modelos

Próxima clase ...

¡Gracias!